SupaDL

Detailed Development Plan & Codex Implementation Blueprint

Version 1.0 | *September 2, 2026*

Python 3.14.x • PySide6 • HTTPX/asyncio • SQLite • Windows-first cross-platform design

Purpose: a reliable, modular download manager inspired by the useful architectural concepts of IDM/JDownloader, built independently and incrementally.

Document Map

1. Executive Summary

SupaDL is a local-first desktop download manager whose central asset is a reusable transfer engine, not a GUI-specific implementation. The first releases focus on correct HTTP/HTTPS transfers, pause/resume, segmented byte-range downloads, persistent queues, retries, checksum verification, and a responsive desktop interface.

Development must be incremental. Each milestone leaves the repository runnable, tested, documented, and safe to continue. Browser interception, host-specific resolvers, and media features are intentionally deferred until the generic engine is trustworthy.

2. Product Principles

- Reliability before speed. A slower correct download is superior to a fast corrupted one.
- Core independent of UI. GUI, CLI, browser extension, and future APIs reuse the same application layer.
- Persistent by default. Interrupted work survives restarts.
- Host-friendly bounded concurrency with Retry-After and backoff.
- Security by default: verify TLS, sanitize filenames, redact secrets, never auto-execute downloads.
- Plugin isolation: site logic belongs outside the transfer engine.
- Observable behavior through typed errors, state transitions, logs, progress and metrics.
- Test against deterministic local fixtures instead of random public websites.
- Respect authorization and protected content. No DRM/access-control bypass.
- Stay pure Python until profiling proves a native component is justified.

3. Scope

3.1 MVP / v0.1-v0.4

- Direct HTTP/HTTPS downloads
- Metadata probing and redirects
- Safe filename/destination handling
- Single-stream and segmented range downloads
- Pause, resume, cancel and restart recovery
- Retries with exponential backoff and Retry-After
- SQLite persistence and queueing
- Progress, speed and ETA
- Optional SHA-256/SHA-512 verification

- PySide6 desktop UI
- Structured logs and typed failures
- Unit/integration/end-to-end tests using local fixture server
- Windows packaging first

3.2 Later Scope

- Browser extension and authenticated loopback/native bridge
- Plugin SDK and link resolvers
- Generic HTML/link grabber
- HLS/DASH support for user-authorized non-DRM content
- FFmpeg remux/merge integration
- Scheduling and bandwidth limiting
- Proxy/authentication profiles
- CLI/headless and optional remote API
- Signed update mechanism

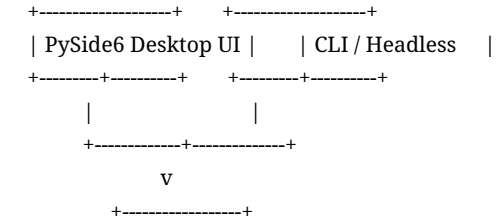
3.3 Explicit Non-goals

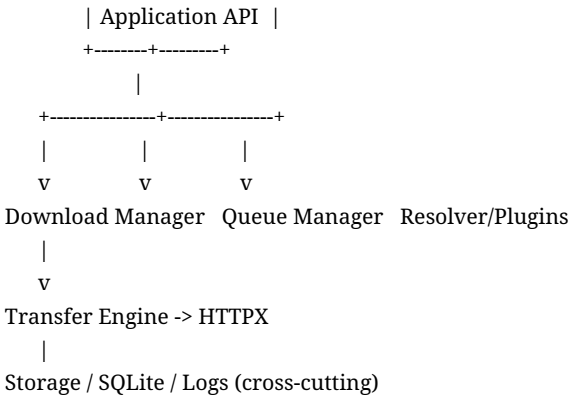
- DRM circumvention
- CAPTCHA or anti-bot bypass
- Credential theft or browser cookie-database scraping
- Paywall/access-control bypass
- Torrent client functionality
- Automatic execution of downloaded programs

4. Recommended Technology Stack

Area	Choice	Reason
Runtime	Python 3.14.x	Current stable feature series; strong async ecosystem
HTTP	HTTPX + asyncio	Streaming, redirects, timeouts, HTTP/2 option
Desktop UI	PySide6 / Qt	Mature cross-platform desktop toolkit
Persistence	SQLite	Transactional, local-first, zero administration
DB wrapper	aiosqlite/repository layer	Async-friendly persistence boundary
Testing	pytest + pytest-asyncio	Strong Python test ecosystem
Lint/format	ruff	Fast, unified developer workflow
Type checking	pyright or mypy	Interface and state safety
Packaging	PyInstaller initially	Pragmatic Windows-first bundling
Media later	FFmpeg	Reuse proven codec/container tooling

5. High-Level Architecture





6. Architectural Layers

Domain

Pure entities, enums, state rules and policies. No Qt, HTTPX, SQLite or filesystem implementation imports.

Application

Use cases such as Add, Probe, Start, Pause, Resume, Cancel, Retry, Verify and recovery.

Infrastructure

HTTP transport, filesystem writer, SQLite repositories, checksum calculator, clock/logging implementations.

Presentation

PySide6 views/models/controllers that call application commands and consume events.

Integrations

Later browser bridge, resolver plugins, media pipeline and external FFmpeg boundary.

6.1 Core Domain Models

Model	Purpose
DownloadTask	Top-level download identity, state and metadata
DownloadSegment	Inclusive byte range and resume position
DownloadSource	Original/final URL and validators
DownloadProgress	Transferred bytes, rate, ETA
ChecksumSpec	Algorithm and expected digest
RetryPolicy	Limits and backoff configuration
DownloadSettings	Per-job overrides
DownloadStatus	CREATED, PROBING, QUEUED, DOWNLOADING, PAUSING, PAUSED, VERIFYING, COMPLETED, FAILED, CANCELLED

7. Repository Layout

```
supadl/
├─ pyproject.toml
├─ README.md
├─ docs/
│  ├─ architecture.md
│  ├─ state-machine.md
```

```
│  ├─ security.md
│  └─ adr/
├─ src/supadl/
│  ├─ domain/
│  ├─ application/
│  ├─ transfer/
│  ├─ persistence/
│  ├─ storage/
│  ├─ security/
│  ├─ plugins/
│  ├─ media/
│  ├─ browser/
│  ├─ ui/
│  ├─ cli/
│  ├─ config/
│  └─ observability/
├─ tests/{unit,integration,fixtures,e2e}/
├─ tools/dev_server/
└─ extension/   # later
```

8. Download State Machine

```
CREATED -> PROBING -> QUEUED -> DOWNLOADING
                        |      |
                        |      +-> PAUSING -> PAUSED -> QUEUED
                        |      +-> VERIFYING -> COMPLETED
                        |      +-> FAILED -> retry -> QUEUED
                        |      +-> CANCELLED
```

- All state transitions go through one application/domain service.
- Transitions are persisted transactionally.
- Crash recovery converts stale DOWNLOADING state into a recoverable PAUSED/QUEUED state according to policy.
- COMPLETED is immutable except history/removal actions.
- Cancellation must stop scheduling new work before temp cleanup.

9. HTTP Probe Strategy

1. Validate http/https URL and policy.
2. Follow redirects within a configured maximum.
3. Attempt HEAD, but never depend on HEAD alone.
4. If HEAD is rejected or inconclusive, use a minimal GET such as Range bytes=0-0.
5. Capture final URL, Content-Length, Content-Type, Content-Disposition, ETag, Last-Modified and range behavior.
6. Treat actual 206 behavior as stronger evidence than Accept-Ranges alone.
7. Derive a sanitized filename and persist validators.

10. Segmented Download Algorithm

Only use segmentation when file size is known and the server demonstrably supports byte ranges. Initial adaptive policy:

File size	Maximum segments
< 8 MiB	1
8-64 MiB	2
64-512 MiB	4
> 512 MiB	8

- Each segment stores inclusive start/end, next byte, downloaded count, state and retries.
- Use Range: bytes=<next>-<end>.
- Require 206 for segmented responses and validate Content-Range.
- If a server stops honoring ranges and sends 200, abort segmented assembly and restart safely under a single-stream policy.
- Earliest prototype should prefer separate segment temp files because debugging and correctness are simpler; random-access writes can follow once tests are mature.

11. Resume Safety

- Persist original/final URL, content length, ETag, Last-Modified, segment offsets and temp paths.
- Re-probe before resume.
- Prefer strong ETag identity. Otherwise use conservative Last-Modified plus length checks.
- If identity changed, return SOURCE_CHANGED instead of appending incompatible bytes.
- Use If-Range where appropriate and validate returned Content-Range.

12. Persistence Schema

Table	Key fields
downloads	id, URLs, filename, paths, size, MIME, ETag, Last-Modified, status, priority, timestamps, bytes, checksum, sanitized error
segments	download_id, index, start, end, next_byte, downloaded, status, retries
settings	typed or key/value application settings
schema_version	migration tracking

Use transactions for lifecycle changes, WAL mode where appropriate, and periodic checkpoint batching instead of database writes for every network chunk.

13. File and Filename Safety

- Parse Content-Disposition defensively, including filename* where practical.
- Strip path separators/control characters and neutralize . / .. semantics.
- Handle Windows reserved names such as CON, PRN, AUX and NUL.
- Apply conservative filename length limits and a deterministic collision policy.
- Keep incomplete data in .supadl.part or an app-private temp location.
- Atomically move/rename on success when the OS/filesystem permits.
- Never automatically launch downloaded content.

14. Retry and Error Model

Category	Default behavior
DNS/connect/read transient failures	Bounded retry with exponential backoff + jitter
HTTP 429	Honor Retry-After, bounded retry
500/502/503/504	Bounded retry
401/403/404	Do not blindly retry
TLS validation failure	Fail; do not disable verification
Disk full/permission	Pause/fail for user action
Invalid range/source changed	Fail safely or restart only under explicit safe policy

15. Queue and Concurrency

- Default maximum active downloads: 3.
- Default maximum connections per download: 4.
- Default maximum connections per host: 8.
- Use per-host semaphores and bounded worker creation.
- Queue manager owns priority/order, starvation prevention, shutdown and aggregate speed.

16. Progress, Speed and ETA

- Byte counters must be monotonic.
- Use a short rolling window or EWMA for displayed speed.
- ETA is shown only when size is known and speed is sufficiently stable.
- Unknown content length uses indeterminate progress.
- Checkpoint progress on time/byte thresholds, not per packet.

17. Desktop UI Specification

Area	Requirements
Toolbar	Add URL, Start, Pause, Resume, Cancel, Remove, Settings
Views	All, Active, Queued, Completed, Failed
Columns	Name, Status, Progress, Size, Speed, ETA, Connections, Destination
Add dialog	URL, resolved name, destination, queue/start, optional checksum, safe advanced options
Details	source/final URL, MIME/size, validators, segments, timestamps, sanitized errors
Threading	No blocking network/disk operations on Qt UI thread

18. Browser Integration (Later)

Browser Extension -> Authenticated loopback/native bridge -> SupaDL application service

- Bind only to loopback.
- Authenticate with a random per-install secret or native messaging.
- Validate origin/request shape and resist CSRF/replay.
- Expose no arbitrary filesystem or shell endpoints.
- Do not scrape browser cookie databases.
- Forward URL/referrer/user-agent/cookies only with browser permission and user-enabled behavior.

19. Plugin Architecture (Later)

```
class ResolverPlugin(Protocol):
    name: str
    priority: int
    def can_handle(self, url: str) -> bool: ...
    async def resolve(self, context: ResolveContext) -> list[DownloadCandidate]: ...
```

- Plugins return normalized candidates; they do not mutate UI/database directly.
- Enforce timeouts and exception isolation.
- Start with built-in plugins only.
- Do not add arbitrary third-party plugin installation until trust/signing/isolation decisions are made.

20. Media Handling (Later)

- Support standards-based HLS and DASH only for content the user is authorized to access.
- Reuse the core transfer scheduler where appropriate.
- Use FFmpeg for remuxing/merging through argument arrays, never shell=True.
- Detect protected/DRM workflows and fail as unsupported rather than attempting circumvention.

21. Security Baseline

- TLS verification enabled.
- Redact Authorization, Cookie, Set-Cookie, signed query tokens and credentials from logs.
- Sanitize all server-provided filenames.
- Use OS credential storage if durable credentials are introduced later.
- Authenticated loopback API only.
- No shell=True external process calls.
- Lock dependencies and scan them in CI.
- Validate signatures before any future auto-update feature.

22. Testing Strategy

22.1 Unit Tests

- segment boundaries
- state transitions
- filename sanitization
- retry policy
- Content-Disposition parsing
- speed/ETA
- checksum verification
- URL/header secret redaction

22.2 Local Fixture Server Integration Cases

- ordinary 200 file
- working 206 ranges
- range advertised but ignored
- HEAD rejected

- redirect chain
- unknown/chunked content length
- disconnect after N bytes
- slow response
- 429 Retry-After
- 500 then recovery
- changed ETag/Last-Modified
- invalid Content-Range

22.3 End-to-End

- add -> complete -> hash match
- pause -> process restart -> resume -> hash match
- segmented -> hash match
- crash recovery
- cancel and cleanup
- disk error surfaced predictably

23. CI Quality Gates

8. ruff check
9. format check
10. type checker
11. pytest unit
12. pytest integration
13. dependency audit where practical
14. packaging/build smoke test

24. Milestone Roadmap

Milestone	Outcome
M0	Repository/foundation
M1	Reliable single-stream downloader
M2	Persistence and resume
M3	Segmented range engine
M4	Queue, retry policies and verification
M5	Desktop GUI
M6	Hardening and packaging
M7	Browser integration
M8	Plugin/link resolver
M9	Standards-based non-DRM media

25. Atomic Codex Task Backlog

SUP-001 - Initialize repository

Objective: Create the production repository skeleton and packaging metadata.

Dependencies: None

Implementation steps:

- Create src/supadl layout, tests directories and docs directories.
- Create pyproject.toml with project metadata and development dependency groups.
- Add README, LICENSE placeholder decision, CHANGELOG and CONTRIBUTING skeletons.
- Provide module entrypoint/CLI help smoke path.

Acceptance criteria:

- Fresh virtual environment can install project.
- Importing supadl succeeds.
- Test command runs successfully.

SUP-002 - Configure quality tools

Objective: Establish repeatable lint, formatting, typing and testing.

Dependencies: SUP-001

Implementation steps:

- Configure ruff.
- Configure pytest and pytest-asyncio.
- Configure pyright or mypy.
- Add documented local quality command(s).

Acceptance criteria:

- All tools run on clean skeleton.
- CI-ready commands are deterministic.

SUP-003 - Typed settings and app paths

Objective: Create safe platform-aware configuration and data locations.

Dependencies: SUP-001

Implementation steps:

- Define settings dataclass/Pydantic settings.
- Resolve config/data/cache/download defaults without hard-coded user paths.
- Add validation for concurrency/timeouts.
- Test path resolution via injectable platform/environment where practical.

Acceptance criteria:

- Defaults validate.
- No unsafe path traversal.
- Settings can be serialized/deserialized.

SUP-004 - Structured secret-safe logging

Objective: Create structured diagnostics that never expose credentials.

Dependencies: SUP-001

Implementation steps:

- Add logging configuration and context fields such as download_id.
- Implement header and URL query redaction helpers.
- Redact Authorization, Cookie and Set-Cookie.
- Add tests for representative secrets.

Acceptance criteria:

- Tests prove secrets are absent from emitted log text.
- Logging can be initialized once without duplicate handlers.

SUP-101 - Domain model foundation

Objective: Define core entities, enums, policies and typed errors.

Dependencies: M0

Implementation steps:

- Create DownloadStatus/SegmentStatus enums.
- Create DownloadTask, DownloadSegment and source/checksum/retry models.
- Define legal transition policy.
- Keep domain free of infrastructure imports.

Acceptance criteria:

- Domain imports without HTTPX/PySide6/SQLite.
- Invalid transitions are rejected.
- Models have stable serialization-friendly primitives.

SUP-102 - Safe filename resolver

Objective: Prevent server-provided names from escaping or breaking destinations.

Dependencies: SUP-101

Implementation steps:

- Parse candidate name from Content-Disposition and URL.
- Sanitize separators/control chars/reserved names.
- Add extension/name fallback.
- Implement deterministic collision naming helper.

Acceptance criteria:

- Traversal and Windows reserved-name tests pass.
- Result is always a basename, never a path.

SUP-103 - HTTP client factory

Objective: Centralize transport lifecycle and safe defaults.

Dependencies: SUP-101

Implementation steps:

- Create async HTTPX client factory/owner.
- Set explicit connect/read/write/pool timeouts.
- Set redirect limit and user-agent.
- Keep TLS verification enabled.
- Expose clean async close lifecycle.

Acceptance criteria:

- Fixture HTTP calls succeed.
- Client closes cleanly.
- No insecure TLS default.

SUP-104 - Probe service

Objective: Discover source metadata and actual range capability.

Dependencies: SUP-102, SUP-103

Implementation steps:

- Implement HEAD attempt.
- Fallback to minimal GET/range probe.
- Capture final URL, length, MIME, filename headers, validators.
- Classify actual 206 range support.

Acceptance criteria:

- HEAD-rejected fixture works.
- Redirect fixture works.
- Misleading Accept-Ranges fixture classified correctly.

SUP-105 - Single-stream worker

Objective: Download a direct URL safely into a temporary file.

Dependencies: SUP-103, SUP-104

Implementation steps:

- Stream bytes in bounded chunks.
- Write to .part path.
- Emit monotonic progress events.
- Flush/close resources on completion or failure.

Acceptance criteria:

- Fixture SHA-256 matches.
- Large response is streamed rather than buffered in memory.

SUP-106 - Cancellation control

Objective: Make active transfers cancellable without corruption.

Dependencies: SUP-105

Implementation steps:

- Define cancellation control passed into workers.
- Check cancellation at safe points.
- Propagate cancellation to coordinator.
- Close network/file handles.

Acceptance criteria:

- Cancellation terminates promptly.
- Partial state remains resumable or clearly marked.

SUP-201 - SQLite bootstrap/migrations

Objective: Create durable schema foundation.

Dependencies: M1

Implementation steps:

- Create DB bootstrap and schema_version.
- Create downloads and segments tables.
- Enable safe pragmas/WAL where supported.
- Add migration transaction wrapper.

Acceptance criteria:

- Fresh DB initializes.
- Reopening is idempotent.
- Schema version is queryable.

SUP-202 - Repositories

Objective: Persist/retrieve download and segment state.

Dependencies: SUP-201

Implementation steps:

- Implement repository interfaces and SQLite implementations.
- Round-trip all relevant fields.
- Use transactions for multi-record changes.

Acceptance criteria:

- Round-trip tests pass.
- Repository consumers do not execute raw SQL.

SUP-205 - Single-stream resume

Objective: Resume partial files using safe byte ranges.

Dependencies: SUP-104, SUP-202

Implementation steps:

- Persist last committed byte.
- Re-probe before resume.

- Request missing range.
- Validate returned range/content identity.

Acceptance criteria:

- Pause/restart/resume yields exact hash.
- Server returning incompatible 200 does not cause append corruption.

SUP-301 - Segment planner

Objective: Split inclusive byte ranges without gaps or overlap.

Dependencies: M2

Implementation steps:

- Implement adaptive segment count policy.
- Generate exact start/end ranges.
- Handle tiny and non-divisible sizes.

Acceptance criteria:

- Property-style tests cover many sizes.
- Union equals [0,size-1] exactly.

SUP-303 - Segment worker

Objective: Download and validate one byte range.

Dependencies: SUP-301, SUP-103

Implementation steps:

- Send Range header.
- Require/parse Content-Range.
- Write only the assigned segment output.
- Checkpoint next byte and retries.

Acceptance criteria:

- Invalid Content-Range fails safely.
- Valid segment bytes hash/length match expected slice.

SUP-304 - Segment coordinator

Objective: Run bounded parallel segment workers.

Dependencies: SUP-303

Implementation steps:

- Use per-download and per-host semaphores.
- Start/stop workers according to lifecycle.
- Aggregate progress.
- Cancel siblings on fatal source identity error.

Acceptance criteria:

- Never exceeds configured concurrency.
- Pause/cancel propagates to all workers.

SUP-305 - Segment assembly

Objective: Produce final exact file and clean temporary segments.

Dependencies: SUP-304

Implementation steps:

- Verify all segments complete.
- Merge in index/order or use validated random-access strategy.
- fsync/close as needed.
- Atomically finalize.

Acceptance criteria:

- Final checksum matches source.
- No temp segment remains after successful cleanup.

SUP-401 - Download manager service

Objective: Provide UI-independent lifecycle orchestration.

Dependencies: M3

Implementation steps:

- Implement application commands for add/start/pause/resume/cancel/retry/remove.
- Centralize transition validation.
- Publish application events.

Acceptance criteria:

- Can be fully exercised in tests without Qt.
- Illegal commands return typed error.

SUP-402 - Priority queue

Objective: Schedule multiple jobs with global active limit.

Dependencies: SUP-401

Implementation steps:

- Create queue abstraction.
- Support reorder/priority.
- Start next eligible task on slot release.

Acceptance criteria:

- Ordering deterministic.
- Global active count never exceeds limit.

SUP-404 - Retry policy engine

Objective: Handle transient network/server failures predictably.

Dependencies: SUP-401

Implementation steps:

- Implement exponential backoff + jitter.
- Honor Retry-After.
- Classify retryable vs terminal failures.
- Make clock/random injectable for tests.

Acceptance criteria:

- 429 and 5xx fixtures obey bounds.
- 401/403/404 are not blindly retried.

SUP-405 - Checksum verification

Objective: Verify completed files on user request.

Dependencies: SUP-401

Implementation steps:

- Implement streaming SHA-256/SHA-512.
- Transition through VERIFYING.
- Record actual digest and mismatch error.

Acceptance criteria:

- Known vectors pass.
- Mismatch never reports COMPLETED.

SUP-501 - PySide6 shell

Objective: Create desktop UI without moving core logic into Qt.

Dependencies: M4

Implementation steps:

- Create application bootstrap and main window.
- Inject application/download manager service.
- Add base toolbar and views.

Acceptance criteria:

- UI launches.
- No network call blocks UI thread.
- Core does not import PySide6.

SUP-503 - Add Download dialog

Objective: Allow URL entry, async probing and destination selection.

Dependencies: SUP-501, SUP-104

Implementation steps:

- Create validated dialog.
- Run probe asynchronously via bridge to app service.
- Display resolved metadata.
- Allow queue/start decision.

Acceptance criteria:

- UI remains responsive during slow probe.
- Invalid URL/probe errors are actionable.

SUP-504 - Lifecycle actions

Objective: Wire user actions to legal application commands.

Dependencies: SUP-501, SUP-401

Implementation steps:

- Bind Start/Pause/Resume/Cancel.
- Reflect state changes via events/model updates.
- Enable/disable actions by legal state.

Acceptance criteria:

- No direct worker manipulation from widget.
- Buttons reflect state correctly.

SUP-601 - Comprehensive fixture server

Objective: Provide deterministic network failure/range scenarios.

Dependencies: M5

Implementation steps:

- Implement endpoints for 200,206,redirect,HEAD rejection,range ignored,disconnect,slow,429,5xx,changed validators,invalid Content-Range.
- Expose fixture metadata/checksum helpers.

Acceptance criteria:

- Integration suite uses fixture server instead of public sites.

SUP-605 - Windows package

Objective: Produce distributable desktop build.

Dependencies: M5

Implementation steps:

- Configure PyInstaller spec.
- Include Qt resources and version metadata.

- Create clean-machine smoke checklist.

Acceptance criteria:

- Packaged executable launches and downloads fixture/public authorized direct file in manual smoke test.

25.1 Remaining Roadmap IDs

Milestone	Remaining task IDs / scope
M2	SUP-203 transitions, SUP-204 checkpoints, SUP-206 source identity validation, SUP-207 startup recovery
M3	SUP-302 range capability, SUP-306 segmented resume, SUP-307 safe fallback
M4	SUP-403 host limiter, SUP-406 collision/final move, SUP-407 metrics
M5	SUP-502 table model, SUP-505 filters, SUP-506 details, SUP-507 settings, SUP-508 tray
M6	SUP-602 crash tests, SUP-603 large-file tests, SUP-604 Windows path tests, SUP-606 versioning, SUP-607 licenses, SUP-608 threat review
M7	SUP-701 through SUP-705 browser bridge/extension security
M8	SUP-801 through SUP-805 resolver plugin SDK and harness
M9	SUP-901 through SUP-906 HLS/DASH/FFmpeg and protected-content detection

26. Definition of Done for Every Codex Task

15. Implement only requested scope.
16. Add or update tests.
17. Run relevant tests and quality tools.
18. Update docs for public behavior/API changes.
19. Avoid unrelated refactors.
20. Report changed files.
21. Report commands/tests and results.
22. List known limitations.
23. Leave repository runnable and green.

27. Codex Working Rules

- Read README, architecture/security docs, pyproject and relevant tests before editing.
- Work on one SUP task or a very small dependency group at a time.
- Do not implement future milestones opportunistically.
- Prefer small composable interfaces over god classes.
- Domain/core must not import PySide6.
- No network activity at import time.
- HTTP client lifecycle must be explicit and closed.
- Never disable TLS verification as a shortcut.
- Never log credentials, cookies or tokens.
- Never use shell=True for external tools.

- Never silently overwrite user files.
- Never trust server filename as a path.
- Handle cancellation explicitly.
- Use bounded concurrency.
- Automated tests use local fixtures, not random public URLs.
- Once users exist, persisted format changes require migrations.

28. Recommended First Codex Prompt

You are implementing SupaDL, a modular Python download manager.

Read the supplied SupaDL master plan completely before changing code.
For this session implement ONLY milestone M0, tasks SUP-001 through SUP-004.

Constraints:
- Python 3.14.x target.
- Use a src/ layout.
- No GUI implementation yet.
- No browser integration, media downloading, plugin host logic, or segmented transfer yet.
- Keep architecture compatible with later async HTTPX and PySide6 layers.
- Configure ruff, pytest, pytest-asyncio, and one type checker.
- Add structured logging with tests proving secrets such as Authorization and Cookie values are redacted.
- Add typed settings and safe platform-specific app data/config paths.
- Keep dependencies minimal.

Before coding:
1. Inspect existing repository state.
2. State which files you plan to create/change.
3. Identify conflicts between repository and master plan.

After coding:
1. Run formatter/linter/type checker/tests.
2. Fix failures caused by your changes.
3. Summarize implementation.
4. List changed files.
5. List commands/test results.
6. List M0 limitations.
Do not begin M1 automatically.

29. Subsequent Codex Prompt Template

Continue SupaDL using the master plan.
Implement ONLY: <TASK IDS>.

Inspect the current repository, relevant architecture/security docs, and tests first.
Do not redo completed tasks unless a failing test or architectural conflict requires correction.

For each task:
- restate acceptance criteria in implementation terms;
- make the smallest coherent change;
- add/update tests;
- preserve security requirements;
- run relevant quality checks.

Do not implement later roadmap items opportunistically.
At the end report changed files, commands/tests, results, and known limitations.

30. First Public Beta Release Gates

- Direct transfer fixture tests pass.
- Pause/process restart/resume yields exact checksum.
- Segmented transfer yields exact checksum.
- Broken/misleading range behavior cannot silently corrupt output.
- Retries and 429 handling are bounded.
- Destination collision behavior is explicit.
- Filename input cannot escape destination.
- Normal logs contain no tokens/auth/cookies.
- UI remains responsive during transfers.
- Shutdown leaves recoverable state.
- Packaged Windows build works on a clean test environment.
- Dependency/license notices and limitations are documented.

31. Future Performance Strategy

Do not assume Python is the bottleneck. Profile network saturation, CPU, disk throughput, SQLite checkpoints, Qt updates and hashing separately. Optimize in this order:

24. Reduce excess UI/progress events.
25. Tune network chunk size.
26. Batch database checkpoints.
27. Optimize file writes.
28. Tune connection concurrency.
29. Evaluate HTTP/2 behavior.
30. Only then consider Rust/native code for a measured hot path.

32. Architecture Decision Records to Create

ADR	Decision
ADR-001	Python 3.14 + asyncio core
ADR-002	Core/UI separation
ADR-003	HTTPX transport abstraction
ADR-004	SQLite persistence/migrations
ADR-005	Segment files vs random-access writes
ADR-006	Browser bridge security model
ADR-007	Plugin trust/isolation model
ADR-008	FFmpeg integration boundary

33. Final Instruction to Codex

Treat this plan as the engineering baseline, not permission to implement everything at once. Work milestone-by-milestone, preserve a green repository, and favor correctness, safe resumability, source validation, security and tests over feature count.

Appendix A - Suggested Developer Commands

```
# environment
python -m venv .venv
# activate environment
pip install -e ".[dev]"

# quality
ruff format .
ruff check .
pytest
# pyright   OR   mypy src

# future CLI examples
supadl add https://example.org/file.iso
supadl list
supadl pause <id>
supadl resume <id>
```

Appendix B - Version Note

This plan targets Python 3.14.x. On September 2, 2026, Python 3.14.7 is the current stable feature release, while Python 3.15.0 remains in release-candidate status with final release scheduled for October 1, 2026. Dependency versions should be resolved and locked during implementation rather than copied blindly from this document.